

Mastering Bash Scripting: Automate Your Repetitive Tasks

Bash scripting turns your boring, repetitive terminal commands into automated superpowers. Learning this early in your career saves you hours of manual work and prevents human errors.

What you'll learn:

- 1 Storing Data with Variables
- 2 Making Decisions with If-Else
- 3 Repeating Work with For Loops
- 4 Checking Success with Exit Status

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Storing Data with Variables

Variables act like named storage boxes in your computer's memory. Instead of hardcoding a server IP or folder name multiple times, you assign it once to a variable. If the server IP changes later, you only have to update it in one single place at the top of your script.

```
$SERVER_IP="192.168.1.50"  
echo "Connecting to $SERVER_IP..."
```

Cloud & DevOps Daily

Bash Scripting: Automate Repetitive Tasks

Saturday, August 01, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Making Decisions with If-Else

Computers are dumb until you give them logic. An if-else statement lets your script check conditions, like whether a file exists or a disk is full. If the check passes, it runs one command; if it fails, it runs another or stops safely.

```
$if [ -d "/var/log" ]; then
    echo "Log directory exists."
else
    echo "Creating log directory..."
    mkdir /var/log
fi
```

Cloud & DevOps Daily

Bash Scripting: Automate Repetitive Tasks

Saturday, August 01, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Repeating Work with For Loops

Never copy-paste code blocks twenty times. A for loop iterates through a list of items—like filenames, server names, or numbers—and runs the exact same set of commands on each item automatically.

```
$for file in *.txt; do  
    echo "Processing $file..."  
done
```

Cloud & DevOps Daily

Bash Scripting: Automate Repetitive Tasks

Saturday, August 01, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Checking Success with Exit Status

Every time a Linux command finishes, it leaves behind a quiet hint called an exit status. A value of zero means everything went smoothly, while any number above zero means an error occurred. You can use this to stop your script if something breaks.

```
$apt-get update
if [ $? -eq 0 ]; then
    echo "Update successful!"
fi
```

Cloud & DevOps Daily

Bash Scripting: Automate Repetitive Tasks

Saturday, August 01, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always start your scripts with the shebang line: `#!/bin/bash`
- + Use descriptive variable names like `BACKUP_DIR` instead of just `d`
- + Quote your variables (like `"$FILENAME"`) to handle spaces in names
- + Add comments liberally to explain the 'why', not just the 'what'
- + Test your scripts with `'bash -n script.sh'` to check for syntax errors

Watch Out For

- ! Forgetting to make your script executable using `chmod +x`
- ! Running untested scripts with root/sudo privileges and breaking the system
- ! Not handling spaces properly in filenames within loops

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh